

MISAKA

A Post-Quantum BlockDAG Layer-1 with a Consensus-Native EVM Execution Lane

The MISAKA Project

github.com/MISAKA-BTC/misakas · Explorer: misakascan.com

Whitepaper v0.1 — June 18, 2026

Abstract. MISAKA is a post-quantum, PQ-only Layer-1 built as a fork of *rusty-kaspa*. It keeps Kaspa’s GHOSTDAG BlockDAG consensus and UTXO ledger but replaces every classically-broken primitive on the consensus surface: transaction authorization moves from secp256k1/Schnorr to ML-DSA-87 (FIPS 204, NIST category 5); the multiplicative MuHash UTXO accumulator becomes the lattice additive LtHash32_1024; and the entire consensus identity (block hash, transaction id, Merkle roots, UTXO commitment, parent references) widens to a 64-byte keyed BLAKE2b-512 digest (*Hash64*). Proof-of-work is a two-layer construction whose consensus-critical Layer 0 finalizer compares against a 512-bit target; the live Layer 1 is a compute-only BLAKE2b-512 || SHA3-512 tag chosen for fast verification. Because a compute-only PoW is no longer ASIC-egalitarian, MISAKA gates deep reorganizations with a second, independent resource: a *DNS-style probabilistic finality overlay* in which staked validators attest to canonical anchors, and a candidate fork that exits a stake-confirmed anchor must dominate the canonical chain in *both* accumulated work and stake. On top of this we run an Ethereum EVM *as part of L1 consensus*—no external bridge, no L2 sequencer—by binding each DAG block’s EVM parent to its GHOSTDAG selected parent and executing transactions under Kaspa-congruent *mergeset delayed acceptance*, so that an EVM result is an append-only function of a block’s parents and a virtual reorganization is a pointer switch rather than a re-execution. We describe the cryptography, consensus, finality overlay, the EVM lane and its UTXO bridge, the native-coin emission schedule, and the planned Fact Settlement Layer, and we state throughout an explicit discipline separating specification-guaranteed values from engineering targets. MISAKA inherits its DAG engine from the Kaspa project, to whom all upstream credit is due.

Contents

1	Introduction	2
2	Preliminaries and Notation	3
3	System Overview	4
3.1	Two lanes over one DAG	4
3.2	Design goals	4
3.3	Non-goals	5
4	Post-Quantum Cryptographic Core	5
4.1	Threat model and the scope of the PQ claim	5
4.2	ML-DSA-87 transaction authorization	6
4.3	Hash64: a 64-byte consensus identity	6
4.4	LtHash32: a lattice UTXO accumulator	6
4.5	Address and the single standard script	7
4.6	Mass and DoS policy	7
4.7	The secp-free guarantee	7

5	Consensus and Proof-of-Work	7
5.1	Inherited DAG consensus	7
5.2	Layered proof-of-work	7
5.3	Width chain and work accounting	8
5.4	Evolution of the Layer 1 tag, and the compute-only trade-off	8
5.5	Difficulty, activation, and re-genesis	9
5.6	Propagation safety	9
6	DNS Probabilistic Finality Overlay	10
6.1	Why a second resource	10
6.2	What the overlay adds	10
6.3	All-active-staker attestation	10
6.4	Quality-gated StakeScore (BFT-free)	10
6.5	The two-dimensional reorg gate	11
6.6	Long-range bound, rollout, and parameters	11
6.7	Operational hardening: validator and remote signer	11
7	The Selected-Parent EVM Execution Lane	12
7.1	The tension, and the selected-parent rule	12
7.2	Mergeset delayed acceptance (the off-by-one)	12
7.3	Commitment structure and version gating	12
7.4	The EVM execution environment	13
7.5	Validity and five-class skip semantics	13
7.6	Gas budgets and two-stage caps	14
7.7	Fee model	15
7.8	Staged scalability	15
7.9	PQ-only integration and secp isolation	16
7.10	Consensus invariants	16
7.11	Performance-claims discipline	16
8	The UTXO $\leftrightarrow$ EVM Bridge	16
8.1	The supply invariant	17
8.2	Deposit (two-stage)	17
8.3	Withdraw (the F002 precompile)	17
9	Native Coin and Emission	17
10	Fact Settlement Layer (Roadmap)	18
11	Implementation and Network Status	19
12	Security Considerations	20
13	Related Work	20
14	Conclusion and Future Work	21

1. Introduction

Two pressures motivate MISAKA. The first is cryptographic: the digital-signature and accumulator primitives that secure essentially every UTXO chain—secp256k1 ECDSA/Schnorr and discrete-log-style accumulators—fall to a sufficiently large quantum computer running Shor’s

algorithm, and the symmetric hashes that anchor chain identity lose half their preimage margin to Grover search. The second is structural: linear-chain settlement is slow, and the BlockDAG model pioneered by Kaspas removes the orphan penalty of high block rates by ordering a *directed acyclic graph* of blocks with the GHOSTDAG protocol, sustaining a high block-per-second (BPS) rate without sacrificing the longest-chain-style security argument.

MISAKA takes the position that these should be solved together, on one base layer, rather than bolted on as overlays to a chain whose core primitives remain classically breakable. Concretely, MISAKA is a **PQ-only** fork of rusty-kaspas: it is not an upgrade path for an existing chain, and it is not wire-, address-, or wallet-compatible with Kaspas or with any prior experimental kaspas-pq state. It is a new network with its own genesis, on which every non-post-quantum path—legacy ECDSA/Schnorr/multisig signatures, legacy addresses, and pay-to-script-hash (P2SH)—is made *unrepresentable* at the consensus, mempool, and wallet layers.

To this PQ base, MISAKA adds two things that distinguish it from a privacy- or payments-only coin. First, a *DNS probabilistic finality overlay*: a proof-of-stake confirmation layer, separate from block production, that lets the network refuse deep reorganizations unless an attacker dominates *both* hashing power and staked value. This two-resource confirmation is what licenses a deliberate change to the proof-of-work—away from a memory-hard, verification-expensive hash toward a compute-only hash that nodes can verify in microseconds, removing the sync bottleneck without making pure-PoW history rewriting cheap. Second, a *consensus-native EVM lane*: a full Ethereum Virtual Machine executed as part of L1 consensus, with no external bridge and no separate sequencer, engineered so that the inherently-sequential EVM coexists with a reorganizing DAG without forcing a replay on every virtual change.

This paper documents the design as it is implemented and frozen in the MISAKA source tree. The treatment is deliberately exhaustive—cryptography, consensus, proof-of-work, finality, the EVM lane, the value bridge, emission, and the planned Fact Settlement Layer—and where a component is a roadmap item rather than active consensus, we say so. Throughout, the format follows the conventions of a technical research note; performance figures are governed by an explicit claims discipline (Section 7.11) that separates what the consensus parameters guarantee from what benchmarks target.

Organization. Section 2 fixes notation and BlockDAG background. Section 3 gives the two-lane architecture and the design goals and non-goals. Section 4 specifies the post-quantum cryptographic core. Section 5 covers consensus and the layered proof-of-work. Section 6 develops the DNS finality overlay. Section 7 is the selected-parent EVM lane, the centerpiece of the protocol. Section 8 specifies the UTXO↔EVM value bridge, Section 9 the native coin and emission, and Section 10 the planned Fact Settlement Layer. Sections 11 to 14 cover implementation status, security considerations, related work, and conclusions.

2. Preliminaries and Notation

We write $\parallel$ for byte-string concatenation and $\text{len}(\cdot)$ for a length prefix. For a keyed hash we write $\text{BLAKE2b-512}(\text{key} = k, m)$ for the BLAKE2b-512 of message m under key k ; MISAKA uses the key slot for *domain separation*, never a string prefix that an attacker could realign. Uint_w denotes an unsigned w -bit integer type. Sizes are in bytes unless stated; one MSK (the native coin) is 10^8 sompi, and one EVM wei is 10^{-10} sompi (Section 8).

BlockDAG and GHOSTDAG. A BlockDAG is a directed acyclic graph $G = (V, E)$ whose vertices are blocks; each block names one or more *parents*, and E points from a block to its parents. The *past* of a block B , $\text{past}(B)$, is the set of blocks reachable from B ; its *anticone* is the set of blocks neither in its past nor its future. GHOSTDAG [1] linearizes G by greedily

selecting, at each step, a k -cluster of mutually well-connected “blue” blocks; it induces a *selected chain* via a per-block *selected parent* $\text{sp}(B)$, and a total order on $\text{past}(B)$. The k parameter is tuned to the network delay so that honest blocks produced within the propagation window are almost always blue. The accumulated proof-of-work along the selected chain is the *blue work*; tip selection maximizes blue work.

Mergeset. For a chain block B the *mergeset* is

$$\text{mergeset}(B) = \text{sp}(B) \cup (\text{anticone}(\text{sp}(B)) \cap \text{past}(B)), \quad (1)$$

i.e. the blocks that B “merges” relative to its selected parent. A fundamental property we use repeatedly (Section 7) is that the mergesets of consecutive selected-chain blocks are pairwise disjoint and their union covers the DAG’s past; this is what makes a per-mergeset accounting rule exactly-once.

UTXO ledger and accumulators. State is a set of unspent transaction outputs (UTXOs). A succinct *UTXO commitment* accumulates the set so that a block header can commit to ledger state in constant size and additions/removals are incremental. Upstream Kasper uses a 3072-bit multiplicative MuHash; Section 4 replaces it.

Confirmation and finality. We distinguish (i) *probabilistic confirmation* against accumulated work (the longest-DAG-style statement), (ii) the DNS overlay’s *stake-confirmed anchor* (Section 6), and (iii) the EVM lane’s RPC finality tags `latest/safe/finalized` (Section 7). These are different objects and we never conflate them.

3. System Overview

3.1. Two lanes over one DAG

MISAKA presents two execution lanes that share a single GHOSTDAG BlockDAG and a single PoW, but are otherwise budget-, gossip-, and I/O-isolated (Figure 1).

- The **UTXO lane** is ML-DSA-87 PQ-only. It uses the existing DAG-inclusive acceptance (a transaction takes effect when its block is ordered into the virtual selected chain’s accepted set) and runs at 10 BPS. Its rules and performance are held invariant by the rest of the design.
- The **EVM lane** is an Ethereum Virtual Machine executed on the selected-parent chain under *mergeset delayed acceptance* (Section 7). It reintroduces secp256k1/ECDSA as an *independent signature domain*, isolated behind a build feature so that the default node binary remains secp-free.

Value moves between the lanes through an in-consensus deposit/withdraw bridge (Section 8) that conserves a single native-coin supply. A second, post-launch consensus layer—the DNS finality overlay (Section 6)—runs orthogonally to both lanes and gates deep reorganizations on a two-resource (work $\times$ stake) rule.

3.2. Design goals

1. **Post-quantum where it matters now.** Make transaction authorization and consensus identity post-quantum, and make the UTXO accumulator lattice-based. Be explicit that transport confidentiality is *not* post-quantum unless an ML-KEM hybrid is enabled.
2. **Keep the DAG engine.** GHOSTDAG block production, blue-work tip selection, the difficulty-adjustment algorithm (DAA), mempool, and short-term confirmation are inherited from Kasper unchanged.

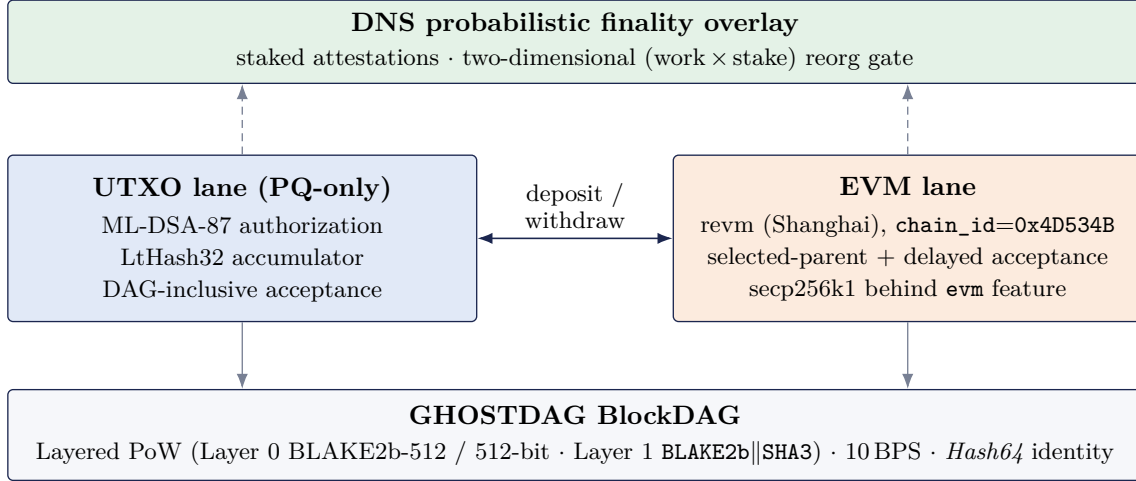


Figure 1. The MISAKA two-lane architecture. One GHOSTDAG BlockDAG and one proof-of-work carry a PQ-only UTXO lane and a consensus-native EVM lane; an in-consensus bridge conserves a single native supply; the DNS finality overlay gates deep reorganizations on a work $\times$ stake rule.

3. **Two-resource deep-reorg resistance.** Add a PoS finality overlay so that rewriting confirmed history requires dominating both work and stake; this is what makes a fast, compute-only PoW acceptable.
4. **EVM as L1 consensus.** Run the EVM with no external bridge and no L2 sequencer, while keeping the UTXO ledger and DAG stability invariant, and keeping “an EVM result is immutable per block hash; a reorg is a pointer switch” true at every stage.
5. **Honest performance accounting.** Separate specification-guaranteed values (derivable from consensus parameters) from engineering targets (benchmark-backed), and never advertise a single hot-object number as general throughput.

3.3. Non-goals

MISAKA does not maintain compatibility with mainline Kaspero addresses, wallets, RPC, or P2P; it does not migrate secp256k1 UTXOs; it does not claim that “all cryptography is post-quantum” (transport is out of scope at launch); it does not ship ML-DSA multisig or P2SH at launch; and it does not claim a single-number throughput crown for fully-re-executed EVM—that target is reserved for the research-stage validity-proof lane (Section 7). The EVM lane is explicitly *not* post-quantum: its signature domain is secp256k1, with the same quantum exposure as Ethereum, and this is disclosed to users.

4. Post-Quantum Cryptographic Core

4.1. Threat model and the scope of the PQ claim

We assume an adversary that will eventually run Shor’s and Grover’s algorithms at scale. Shor breaks the discrete-log and factoring assumptions, which voids secp256k1 signatures and the RSA-style group underlying MuHash. Grover gives a quadratic speed-up on preimage and collision search against symmetric hashes, halving their effective margin. MISAKA’s PQ claim is precise and bounded:

Transaction authorization uses ML-DSA-87; secp256k1 signing is disabled in PQ consensus mode; the consensus identity is a 64-byte BLAKE2b-512 digest; the UTXO accumulator is lattice-based. Transport-layer confidentiality is *not* post-quantum unless an ML-KEM hybrid key exchange is enabled (out of scope at launch). No claim is made that “all cryptography is post-quantum,” and legacy Kaspero addresses are explicitly *not* quantum-resistant.

Identity	Type	Producer
Block hash	BlockHash=Hash64	keyed BLAKE2b-512 over the header preimage
Transaction id	Hash64	keyed BLAKE2b-512 over the tx preimage
Merkle roots	Hash64	hash-Merkle / accepted-id Merkle, 64-byte nodes
UTXO commitment	UtxoCommitment64	LtHash32 finalize (Section 4.4)
Pruning point, parents	Hash64	block-hash references

Table 1. Consensus identities widened to 64 bytes / 512 bits (keyed BLAKE2b-512). EVM-internal roots stay keccak-256/32-byte for Ethereum-tooling compatibility.

4.2. ML-DSA-87 transaction authorization

Signatures are ML-DSA-87 (FIPS 204, the standardized Module-Lattice DSA, NIST security category 5), via a pinned `libcrux-ml-dsa` ($\geq 0.0.9$, exact-pinned with lockfile audit on the consensus path). Parameter sizes are public key 2592, secret key 4896, and signature 4627 bytes; the signature element carried on a script is `signature || sighash_type = 4628` bytes. Domain separation uses the context string `kaspa-pq-v2/tx/mldsa87`.

The signature transcript is a dedicated 64-byte sighash, *not* the legacy 32-byte Schnorr sighash, which would be weak in both scheme and width:

$$\text{sighash}_{\text{mldsa87}}(\text{tx}, i, \text{type}) = \text{Hash64}(\text{semantic fields} \parallel \text{domain kaspa-pq-v2/sighash/mldsa87} \parallel \text{class tag}), \quad (2)$$

where the script-class tag binds the standard ML-DSA-87 P2PKH class, and every constituent hash (previous outputs, sequences, sig-op counts, outputs, payload) is itself widened to Hash64. The verifier passes the 64-byte digest directly into the ML-DSA-87 verify routine under the transaction context; no `secp256k1::Message` appears on the PQ path. A signature-verification cache keys on three BLAKE2b-512 digests of (public key, signature, message) under an algorithm tag, so raw keys and signatures are never retained.

4.3. Hash64: a 64-byte consensus identity

Even with a 512-bit PoW finalizer (Section 5) and a widened UTXO commitment, leaving the block hash, transaction id, Merkle roots, pruning point, and parent references at 256 bits would cap their preimage cost at 2^{128} under Grover and re-introduce a quantum-margin asymmetry. MISAKA therefore makes the *entire* consensus-visible identity 64 bytes / 512 bits, produced by keyed BLAKE2b-512 over the relevant preimage (the *Hash64* type). Table 1 lists the migrated identities. Because MISAKA is a new network, the one-time wire- and storage-format cost is paid up front rather than retrofitted onto a live chain. A deliberate exception preserves Ethereum tooling: *inside* the EVM lane, state/receipts/transactions roots remain keccak-256/32-byte (EvmH256); only the L1 *consensus commitment* over EVM execution is Hash64 (Section 7).

4.4. LtHash32: a lattice UTXO accumulator

The MuHash accumulator is a multiplicative group of 3072-bit residues; its security rests on a discrete-log-style assumption that Shor breaks. MISAKA replaces it with *LtHash*, an additive accumulator whose security reduces to lattice (short-integer-solution) problems and whose add/remove are the same component-wise operation with an inverse element. After an audit revision MISAKA uses LtHash32_1024: 1024 lanes of 32 bits (a 4096-byte state, ≈ 170 -bit quantum collision margin), upgraded from the original 16-bit-lane variant whose ≈ 100 -bit quantum margin was the weakest link. The accumulator is a commutative group under component-wise addition modulo 2^{32} ; the empty-state finalize is a 64-byte BLAKE2b-512, sharing the keyed-BLAKE2b family used throughout.

4.5. Address and the single standard script

The only address version is `PubKeyHashMlDsa87`; its payload is a keyed BLAKE2b-512 of the verification key, 64 bytes:

$$\text{payload} = \text{BLAKE2b-512}(\text{key} = \text{kaspa-pq-v2/address/mldsa87}, vk).$$

Mainnet addresses begin `misaka`, testnet `misakatest`, devnet `misakadev`. The only standard script is an ML-DSA-87 pay-to-public-key-hash:

```
OP_DUP OP_BLAKE2B_512 OP_DATA64 <payload64> OP_EQUALVERIFY OP_CHECKSIG_MLDSA87
```

P2SH is disabled: not merely de-prioritized, but rejected before the script branch executes, so that a legacy signature opcode cannot be smuggled back through a redeem script. The legacy signature opcodes (`OP_CHECKSIG`, `OP_CHECKSIGECDSA`, `OP_CHECKMULTISIG` and verify variants) are consensus failures in PQ mode. Crucially, these checks live in the *consensus validator and script engine*, not only in the mempool, so a miner that injects a block directly cannot bypass them. ML-DSA multisig is out of launch scope; if added later it will be a dedicated statically-analyzed class, never a blanket P2SH re-enable.

4.6. Mass and DoS policy

ML-DSA-87 objects are large, so the mass model is recalibrated to price signature verification and bound block compute. The signature-operation mass is `mass_per_sig_op` = 10,000; the maximum script element is 8192 bytes (a 4628-byte signature and a 2592-byte key both fit); and the script and signature-script length caps are 16,384 bytes (an ML-DSA P2PKH unlock is ≈ 7.3 KB). Dust thresholds are computed from the ML-DSA P2PKH spend size rather than the upstream 148-byte assumption. The compute-mass limit caps a block at roughly 197 ML-DSA transactions.

4.7. The secp-free guarantee

In the default build the `secp256k1` dependency is feature-gated out of both `kaspa-consensus` and the `kaspad` node binary (default feature `pq-only`); legacy opcode implementations, legacy wallet APIs, and legacy tests live behind a `legacy-secp256k1` feature used only by non-node tooling. A continuous-integration guard (`scripts/pq-ci-guard.sh`) hard-fails the build if a dependency tree edge re-introduces `secp256k1` into the consensus crate or the node, and the same guard covers the nine production binaries. The EVM lane’s `secp256k1` (Section 7) is reintroduced only behind a separate `evm` build feature (default off).

5. Consensus and Proof-of-Work

5.1. Inherited DAG consensus

Block production, tip selection, work ordering, and difficulty are inherited from Kasper and unchanged by the PQ migration: blocks are produced by PoW; GHOSTDAG selects tips; the work order is the *blue work* accumulator; and the difficulty-adjustment algorithm is blue-work-driven. MISAKA runs every network at 10 BPS—`target_time_per_block` = 100 ms—with the GHOSTDAG cluster parameter k computed from the network delay bound and block rate by the standard rule $k = \text{GHOSTDAG-K}(2\lambda D, \delta)$, where λ is the block rate and D the propagation delay.

5.2. Layered proof-of-work

Upstream Kasper’s PoW conflates three concerns: the comparison domain (the integer width the miner solves against), the ASIC-resistance function (the heavy hash the miner runs), and

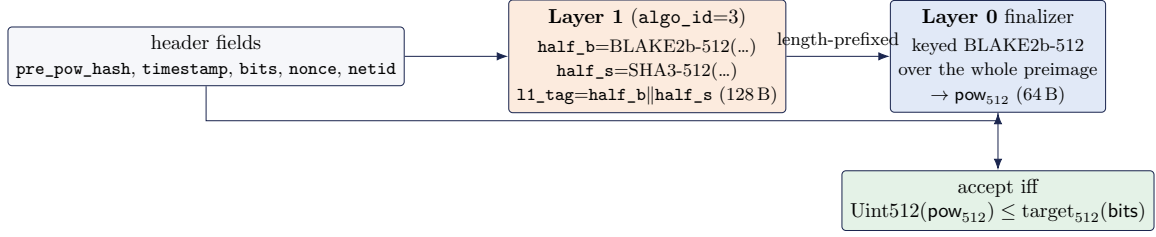


Figure 2. The two-layer PoW. The compute-only Layer 1 tag (BLAKE2b-512 || SHA3-512, `algo_id=3`) is length-prefixed into the consensus-critical Layer 0 BLAKE2b-512 finalizer, whose 512-bit digest is compared against a 512-bit target. Layer 0 also re-absorbs the raw header fields, so a miner cannot skip the SHA3 half.

the work-accounting width. MISAKA separates them so each can move on its own hard-fork schedule. The PoW is a two-layer construction (Figure 2):

$$\begin{aligned}
 \text{L1_tag} &= \text{AsicHardFn}_{\text{algo_id}}(\text{pre_pow_hash}, \text{timestamp}, \text{bits}, \text{nonce}) \\
 \text{pow}_{512} &= \text{BLAKE2b-512}\left(\text{key} = \text{kaspa-pq-pow-v1}, \text{netid} \parallel \text{algo_id} \parallel \text{pre_pow_hash} \parallel \text{timestamp} \right. \\
 &\quad \left. \parallel \text{bits} \parallel \text{nonce} \parallel \text{len}(\text{L1_tag}) \parallel \text{L1_tag}\right) \\
 \text{accept} &\iff \text{Uint512}(\text{pow}_{512}) \leq \text{target}_{512}(\text{bits}).
 \end{aligned} \tag{3}$$

Layer 0 — the quantum-resistant finalizer (consensus-critical). A keyed BLAKE2b-512 (HAIFA construction) producing 64 bytes compared against a 512-bit target. It is keyed with `kaspa-pq-pow-v1` and is self-delimiting: the preimage embeds `network_id` and an explicit length prefix on the Layer 1 tag, so a future `algo_id` variant cannot collide with a prior encoding. The Layer 0 family is deliberately *different* from the Keccak/SHA-3 family used by Layer 1, so a structural break in one family does not pass through both halves of the same PoW. Layer 0 is frozen and never changes without an additional hard-fork ADR.

Layer 1 — the ASIC-resistance tag. Identified by an 8-bit `algo_id` in the header. Switching is a hard cut-off: two `algo_id` values are never simultaneously valid at the same DAA score, and there is no mixed-algorithm difficulty arithmetic.

5.3. Width chain and work accounting

The widths cascade so that work never overflows (Table 2). A compact 32-bit `bits` field expands to a 512-bit target; the finalizer yields a 512-bit digest; single-block work is $\text{work} = \lfloor 2^{512} / (\text{target} + 1) \rfloor$ held in a 576-bit accumulator (one machine word wider than the target, so a window of 2^{64} maximum-work blocks still fits); and the DAA window arithmetic uses 640 bits. The difficulty-lift identity $\text{target}_{512} = \text{target}_{256} \ll 256$ preserves block-finding probability exactly, $\Pr[X_{512} \leq \text{target}_{256} \ll 256] = \text{target}_{256} / 2^{256}$, and serves as documentation of the 256-to-512-bit map rather than a migration path (MISAKA is a fresh chain).

5.4. Evolution of the Layer 1 tag, and the compute-only trade-off

The Layer 1 algorithm has had three phases. **Phase 1** (`algo_id=1`) re-exported the upstream `kHeavyHash` matrix; it made no ASIC-resistance claim. **Phase 2** (`algo_id=2`) shipped a memory-hard Argon2id (16 MiB) tag to compress the GPU-to-ASIC gap. It worked, but exposed a structural cost: a memory-hard PoW is *symmetric*—the verifier pays a large fraction of the miner’s per-hash cost. Measured on the weakest mesh host, one Argon2id header

Quantity	Type	Width	Note
bits	compact	32-bit	header-encoded target
PoW target / digest	Uint512	512-bit	Layer 0 comparison domain
Single-block work	Uint576 (BlueWorkType)	576-bit	$\lfloor 2^{512}/(\text{target} + 1) \rfloor$
DAA window arithmetic	Uint640	640-bit	aggregation headroom

Table 2. PoW width chain. The accumulator is one word wider than the target as a deliberately impossible-to-reach safety ceiling.

verification is ≈ 48 ms, which makes PoW verification the initial-block-download bottleneck, since a syncing node spends most of its CPU re-running Argon2id.

Phase 3 (`algo_id=3`), the active algorithm, replaces Argon2id with a compute-only tag:

$$\begin{aligned}
 \text{half_b} &= \text{BLAKE2b-512}(\text{key} = \text{kaspa-pq-11-blake2b-sha3-v1}, \text{len}(\text{netid}) \parallel \text{netid} \parallel \text{pre_pow_hash} \parallel \text{nonce}), \\
 \text{half_s} &= \text{SHA3-512}(\text{kaspa-pq-11-blake2b-sha3-v1} \parallel \text{len}(\text{netid}) \parallel \text{netid} \parallel \text{pre_pow_hash} \parallel \text{nonce}), \\
 \text{ll_tag} &= \text{half_b} \parallel \text{half_s} \quad (128 \text{ bytes}).
 \end{aligned} \tag{4}$$

The 128-byte tag feeds the unchanged Layer 0 finalizer, which mixes a second BLAKE2b-512 over the *whole* preimage (including `half_s`); a miner therefore cannot skip the SHA3 half. Per-nonce work is $2 \times \text{BLAKE2b-512} + 1 \times \text{SHA3-512}$, giving roughly a $10^4 \times$ faster verification ($\approx \mu\text{s}/\text{header}$).

The trade-off is explicit and accepted: a compute-only PoW is not memory-hard, so GPU/FPGA/ASIC acceleration is possible, and Phase 3 gives up the kHeavyHash/Argon2id ASIC-resistance goal. This is acceptable *only* because PoW is no longer the sole security pillar. The two-dimensional DNS finality overlay (Section 6) gates reorganizations on stake-confirmed anchors, so a pure-PoW majority cannot rewrite confirmed history. The decision optimizes for sync/verification cost—a real operational pain—over PoW egalitarianism, which the overlay already softens. Concatenating two hash families is a cryptographic hedge: a break in either BLAKE2b-512 or SHA3-512 still leaves the Layer 0 comparison intact.

5.5. Difficulty, activation, and re-genesis

Phase 3 is gated by a per-network activation predicate (always-on for testnet/mainnet, never for devnet/simnet, which stay kHeavyHash), and `check_algo_id` enforces the exact admitted id per header so the single-algorithm invariant holds. Switching the algorithm alone would leave the (PoW-exempt) genesis hash byte-identical and risk an un-wiped node silently grafting Phase-3 blocks onto Phase-2 history; MISAKA therefore re-generises with a Phase-3 coinbase marker ("`-bs3`") that changes the live networks' genesis hashes, plus a startup genesis-mismatch guard that refuses to open a database whose recorded genesis differs from the configured one. Inherited `bits` are an easy launch floor; the DAA ramps difficulty to the hash-rate equilibrium $D \approx (\text{aggregate H/s})/\text{BPS}$ within the first difficulty window under un-throttled mining and never stalls.

5.6. Propagation safety

A high BPS is safe only while the GHOSTDAG inclusion window absorbs propagation delay. The governing inequality is

$$\lambda \cdot D_{\max} \lesssim k, \tag{5}$$

relating block rate λ , the worst-case propagation delay $D_{\max}$, and the cluster parameter k . Violating it splits the DAG: an early testnet incident traced a DAG split to geographic propagation delay breaching Equation (5), resolved by enforcing a minimum block interval.

Because the EVM lane adds bytes to blocks and thus raises $D_{\max}$, re-validating Equation (5) with payload-inclusive measured delay is a precondition for EVM activation (Section 7).

6. DNS Probabilistic Finality Overlay

6.1. Why a second resource

Pure PoW—even quantum-hardened against single-block grinding—depends on a single resource, hashing power, so a PoW-majority adversary can in principle deep-reorganize arbitrarily far. MISAKA adds a *DNS-style* (Double-Nakamoto-Score-style) finality overlay: a proof-of-stake confirmation layer, separate from block production, whose value is *non-substitutability*. History replacement requires dominating both work and stake; a surplus in one resource does not compensate for a deficit in the other. PoW/GHOSTDAG continue to drive block production and tip selection unchanged; the overlay only adds on-chain attestation objects, a deterministic *StakeScore*, and one reorg rule.

6.2. What the overlay adds

Four transaction-carried object families, plus parameters, a read view, and a rule:

- **StakeBond** — locks coins to an ML-DSA-87 validator key, with an activation DAA score and an unbonding period satisfying the long-range bound below.
- **StakeAttestation** — one ML-DSA-87 signature by a validator over the tuple (epoch, selected-chain anchor, validator-set commitment, bond outpoint).
- **StakeAttestationShard** — a transaction carrying 8–16 attestations, capped per block; multiple shards across multiple blocks reconstruct an epoch certificate, so no single huge-certificate transaction is needed.
- **SlashingEvidence** — two incompatible attestations from the same (bond_outpoint, validator, epoch); submitting it within the evidence window burns the bond.

6.3. All-active-staker attestation

MISAKA does *not* use a sortition committee. The participation model is *stake* $\rightarrow$ *participate*: every validator holding an active stake bond attests every epoch, so there is no per-epoch committee subset, no sortition, and no commit-reveal randomness pipeline. This reverses an earlier committee design; inspection of the implemented pipeline established that the committee was never a block-validity input—each consensus attestation-verification site verifies the ML-DSA-87 signature over a message containing the attestation’s own self-declared validator-set commitment, so removing sortition removes machinery without removing a consensus check. At 10 BPS an attestation epoch (attestation_epoch_length_blue_score= 100) is only ≈ 10 s of wall-clock, so a single validator polling every few seconds stays current.

6.4. Quality-gated StakeScore (BFT-free)

The naive StakeScore credit is linear and ungated: an epoch earns $\text{credit}_e = (\text{included_stake}_e / \text{expected_stake}_e) \times \text{SCALE}$, so even a small included fraction earns a small positive credit every epoch, and over a long window a *minority* stake could accumulate score and edge a chain toward confirmation. MISAKA adds a quality floor φ_S : below it an epoch earns *zero* StakeScore. This is deliberately *not* a BFT two-thirds supermajority—the PoS layer is not a BFT finality gadget—but a quality gate that denies long-run accumulation to a minority. The overlay remains BFT-free: PoW/GHOSTDAG is the sole block-production and tip-selection consensus, and StakeScore is only a second, independent confirmation dimension.

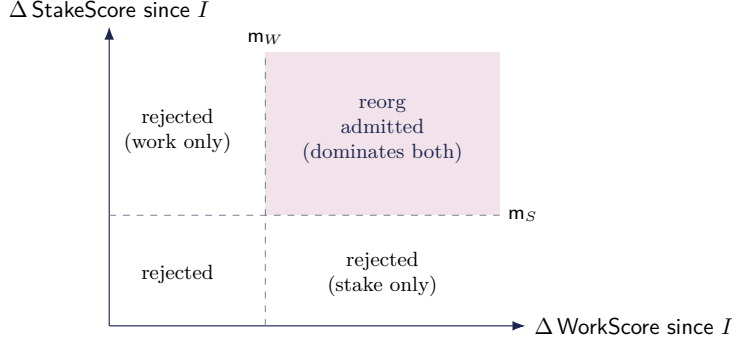


Figure 3. The two-dimensional dominance gate. A candidate fork that exits a stake-confirmed anchor is admitted only in the upper-right region, where it beats the canonical chain on *both* accumulated work and accumulated stake by the margins m_W, m_S . Neither axis alone suffices.

6.5. The two-dimensional reorg gate

Let I be the common ancestor of a candidate fork and the canonical tip, and let $W(\cdot | I)$ and $S(\cdot | I)$ be the work- and stake-score accumulated since I . A candidate that does not contain the latest stake-confirmed anchor is admitted only if it dominates on both dimensions (Figure 3):

$$(\text{cand} \not\subseteq \text{confirmed anchor}) \Rightarrow \left[W_{\text{cand}} > W_{\text{canon}} + m_W \wedge S_{\text{cand}} > S_{\text{canon}} + m_S \right], \quad (6)$$

otherwise the reorganization is rejected (`DnsDominanceViolation`). The margins m_W, m_S are consensus parameters set so that overcoming both at once is exponentially less likely than overcoming either alone. Two implementation notes are stated honestly. First, the shipped production presets set the confirmation work-depth threshold $c_W = 0$, so the confirmation *predicate* reduces to a *stake-confirmed canonical lagged anchor*; the PoW non-substitutability is enforced by the *reorg gate* Equation (6) (which reads deltas since I), not by the predicate (which reads cumulative work). The accurate public description is therefore “stake-confirmed canonical anchor + two-dimensional reorg gate,” not “Double-Nakamoto confirmation.” Second, making work-depth a genuine confirmation input requires both $c_W > 0$ and a change from cumulative blue work to anchor-relative depth.

6.6. Long-range bound, rollout, and parameters

PoS introduces a long-range surface that PoW lacks; MISAKA bounds it with a consensus rule on the unbonding period, $U \geq R + E$, where R is the maximum reorg horizon and E the evidence window, so a bond cannot be withdrawn before any equivocation it signed can still be slashed. The overlay cannot be enabled at genesis (no stake exists), so the rollout is three-stage: (i) a *launch* phase that is PoW-only and rejects bond/attestation transactions; (ii) a *bootstrap* phase that accepts bonds and attestations for visibility but does not enforce the gate; and (iii) an *activation* phase, engaged once total active stake and active-validator count clear their minimums and the activation DAA score is reached, after which Equation (6) is enforced. The production presets (testnet/mainnet) require two-dimensional confirmation and a minimum stake bond of 20,000,000 MSK (2×10^{15} sompi); the RPC `getDnsConfirmation` reports `dnsConfirmed` and a `lastDnsConfirmedAnchor`, and can answer per-block finality for a supplied block hash.

6.7. Operational hardening: validator and remote signer

A `kaspa-pq-validator` sidecar connects to a local node over WebSocket-RPC and attests while its bond is active. For key isolation, `kaspa-pq-signer` is a standalone daemon that holds the validator key *outside* the validator process, answering sign requests over an owner-only (0700)

Unix-domain socket under a configurable policy (`permissive/audit-only/strict`), a strict-mode anti-equivocation guard backed by a crash-consistent signed-epoch store, and a tamper-evident hash-chained audit log, so that a compromised validator node can neither exfiltrate the key nor double-sign.

7. The Selected-Parent EVM Execution Lane

7.1. The tension, and the selected-parent rule

We run the Ethereum EVM as part of L1 consensus—no external bridge, no L2 sequencer. The tension is that the EVM is a global mutable state machine, inherently sequential, whereas a DAG accepts many blocks in parallel and reorganizes its virtual selected chain frequently; re-executing every accepted block’s EVM transactions on each virtual change would force a perpetual reorder/replay. MISAKA resolves this by binding the EVM parent of a DAG block B to its GHOSTDAG *selected parent*—not its full parent set, and not the current virtual selected parent:

$$\begin{aligned} \text{EVM_PARENT}(B) &= \text{sp}(B), \\ \text{EVMRESULT}(B) &= \text{EXECUTE EVM} \left(\underbrace{\text{EVMSTATEROOT}(\text{sp}(B))}_{\text{parent state}}, \underbrace{B.\text{system_ops}}_{B\text{-scoped}}, \right. \\ &\quad \left. \underbrace{\text{ACCEPTEDEVMTXS}(B)}_{\text{from mergeset}(B)}, \underbrace{\text{EVMBLOCKENV}(B)}_{\text{env, § 5}} \right). \end{aligned} \quad (7)$$

The single consequence that organizes the rest of the design: **EvmResult(B) is a function of B ’s parents and B ’s own system operations alone.** It is computed once and stored under B ’s block hash; a virtual reorganization never re-executes it. The EVM head is moved by switching a canonical pointer (`latest/safe/finalized`), and there is no `execute/revert` on the hot path.

7.2. Mergeset delayed acceptance (the off-by-one)

User EVM transaction *inclusion* is open to every DAG block; *execution* is defined as a chain block’s mergeset acceptance, exactly mirroring UTXO acceptance:

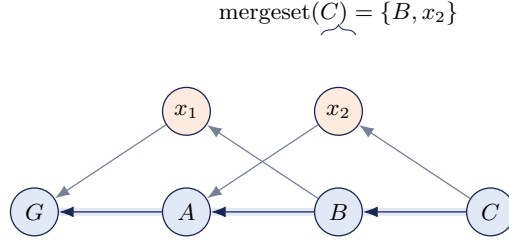
$$\text{ACCEPTEDEVMTXS}(B) = \text{concat}_{X \in \text{sorted mergeset}(B)} (X.\text{evm_payload.transactions in payload order}). \quad (8)$$

Crucially, B ’s own user payload is not in $\text{EVMRESULT}(B)$; it is accepted by B ’s selected child. This is the principled resolution of the off-by-one. Correctness rests on mergeset disjointness (Equation (1)): consecutive selected-chain mergesets are pairwise disjoint and cover the past, so every payload is executed *exactly once*—no double execution, no omission (Figure 4). A side benefit: because $\text{ACCEPTEDEVMTXS}(B)$ depends only on B ’s parent choice, a producer can finish acceptance execution and commitment *before* choosing its own payload, decoupling EVM execution from PoW grinding and payload selection.

System operations (deposit claims, Section 8) are the exception: they remain in B ’s own payload and execute in B , because they are producer-selected and validated against the selected-parent’s UTXO view. The execution order within a block is system operations first, then accepted user transactions.

7.3. Commitment structure and version gating

Delayed acceptance requires a block to commit *two* things separately: its own payload (data) and the result of executing its mergeset (execution). Two 64-byte header fields, both keyed



selected chain (bold): $G \rightarrow A \rightarrow B \rightarrow C$. Blue = chain blocks, orange = merged anticone.

$\text{ACCEPTEDEVMTXS}(C)$ executes the payloads of $\text{mergeset}(C)$; C 's own payload waits for C 's selected child.

Figure 4. Mergeset delayed acceptance on a BlockDAG. A chain block executes the EVM payloads in its mergeset (its selected parent plus the merged anticone), and its own payload is executed one step later by its selected child. Disjoint, covering mergesets make every payload execute exactly once.

BLAKE2b-512 over Borsh, do this:

$$\begin{aligned} \text{evm_payload_hash}(B) &= \text{BLAKE2b-512}(\text{key} = \text{EvmPayload64}, \text{Borsh}(B.\text{evm_payload})), \\ \text{evm_commitment_root}(B) &= \text{BLAKE2b-512}(\text{key} = \text{EvmCommitment64}, \text{Borsh}(\text{EVMEXECUTIONHEADER}(B))). \end{aligned} \quad (9)$$

The execution header carries the EVM state/transactions/receipts roots, gas used, base fee, EVM block number and timestamp, the coinbase, accepted and skipped transaction counts, an $O(1)$ total-native-balance accumulator for the supply invariant, and a base-fee burn accumulator. The two-layer hash convention is the formal specification: EVM-internal roots are keccak-256/32-byte; L1 consensus commitments are Hash64/keyed BLAKE2b-512.

The header fields exist always (zero by default) but enter the header-hash preimage only when $\text{header.version} \geq \text{EVM_HEADER_VERSION} = 2$. Genesis is version 0 and pre-EVM mined blocks are version 1, both < 2 , so every genesis hash and pre-EVM block identity is byte-for-byte unchanged—the EVM lane is a version-gated hard fork that does not disturb history.

7.4. The EVM execution environment

Table 3 lists the frozen parameters. The initial fork is revm's `SpecId::SHANGHAI` (PUSH0 and current Solidity output; Uniswap v2/v3 run); Cancun/EIP-1153 transient storage is a later, separately-activated fork, and there is no automatic upstream tracking (a fork bump is a hard fork). The EVM block number increments along the selected chain, $\text{evm_number}(B) = \text{evm_number}(\text{sp}(B)) + 1$. To avoid coupling EVM logical time to BPS, the timestamp clamp is *non-decreasing* rather than strictly monotone,

$$\text{evm_timestamp_sec}(B) = \max(\text{header_ts}(B), \text{evm_timestamp_sec}(\text{sp}(B))),$$

so multiple consecutive EVM blocks may share a timestamp; developers are directed to use `block.number` for sequencing and `block.timestamp` only as lower-resolution wall-clock time, which keeps Uniswap v2/v3 TWAP oracles correct (they skip or early-return on equal timestamps). To avoid a cyclic dependency—the header hash commits the EVM result, so it cannot be an EVM input—`prevrandao` and `blockhash` are derived from the selected-parent ancestry only.

7.5. Validity and five-class skip semantics

Under delayed acceptance the accepting producer did not choose the payload contents, so a payload's defects cannot be allowed to invalidate the accepting block. MISAKA classifies transaction outcomes into five classes (Table 4). The key distinctions: a *payload-admission* violation (class 1) is a cheap syntactic check that invalidates the *payload* block at body validation

Parameter	Value	Note
EVM_HEADER_VERSION	2	genesis v0 / pre-EVM v1 stay < 2
EVM_CHAIN_ID	0x4D534B (“MSK”)	EIP-155; disjoint from Ethereum nets
EVM fork	revm SpecId: :SHANGHAI	Cancun is a later fork; no auto-tracking
EVM_NATIVE_SCALE	10^{10}	sompi (8 dp) $\rightarrow$ wei (18 dp)
EIP-1559	elasticity 2, denom 8, base fee burned	initial base fee 1 gwei
WMISAKA predeploy	0x...F001	WETH9-compatible wrapped native
Withdraw precompile	0x...F002	Section 8
ML-DSA-87 verify precompile	0x...F003	FSL only (Section 10)
Execution gas cap	G_{limit} per chain block	per-second-derived (§ 7.6)

Table 3. Frozen EVM-lane parameters. The chain id “MSK” is chosen disjoint from public Ethereum networks; the mainnet final value is set at launch.

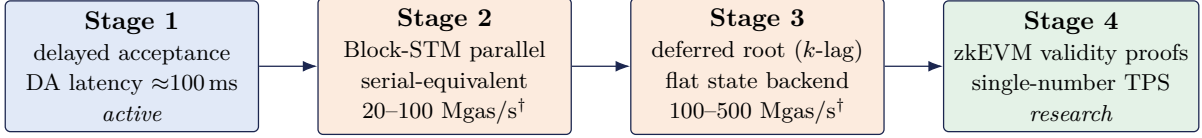
Class	Condition	Handling	Blame
1	malformed encoding/signature, chain-id mismatch, below fixed intrinsic floor, over-size	payload block invalid (body validation)	payload producer
2	bad nonce, balance $<$ upfront cost, max-fee $<$ base fee(B)	deterministic skip; no receipt; nonce unchanged	none
3	duplicate tx hash	auto-skip via class-2 nonce rule	—
4	revert, OOG, precompile/slippage/hook failure	executed; receipt status 0; gas charged	user
5	accepted-gas-cap overflow (canonical tail)	deterministic prefix skip; nonce unchanged	—

Table 4. Five-class skip semantics. Only class 1 (a payload block’s own syntactic defect) and the commitment/system/UTXO checks can invalidate a block; the rest are skips invisible to the EVM result.

(its own producer’s fault); an *acceptance-time* skip (class 2: bad nonce, insufficient balance, max-fee below the block’s base fee) is a deterministic skip with no receipt, no gas charge, and an unchanged nonce, blamed on no one because the base fee was unknown when the payload was built; a *duplicate* (class 3) is absorbed automatically by the class 2 nonce rule, needing no seen-set; an *execution failure* (class 4: revert, out-of-gas, slippage) is executed with a status-0 receipt and gas charged, the user’s fault; and a *gas-cap overflow* (class 5) is a deterministic prefix skip. Classes 2/3/5 are invisible to the EVM result—state, receipts, and gas used are unaffected—and a skipped class-2/5 transaction, with its nonce unchanged, is naturally re-accepted by a later block once conditions hold. A block becomes invalid or disqualified-from-chain only on a commitment/root mismatch, a system-op consensus violation, a UTXO-diff inconsistency, or a class-1 syntactic violation of its own payload.

7.6. Gas budgets and two-stage caps

A per-DAG-block payload-byte cap alone would let a chain block that merges a large mergeset have an unbounded execution budget, so MISAKA uses two caps: `MAX_EVM_PAYLOAD_BYTES_PER_DAG_BLOCK` (inclusion side, checked in body validation) and `MAX_EVM_ACCEPTED_GAS_PER_CHAIN_BLOCK` (execution side, equal to the per-second-derived gas limit). The execution cap is applied as a deterministic prefix-take on `gas_limit`: `ACCEPTED_EVM_TXS( $B$ )` is scanned in canonical order and every transaction from the first whose cumulative gas limit exceeds the cap is class-5 skipped. Judging on `gas_limit` fixes the accepted set *before* execution, giving the parallel



[†] engineering targets, low-mid contention; governed by the claims discipline of Section 7.11.

Figure 5. The four-stage EVM scalability roadmap. Stage 1 is active; Stages 2–3 are implementation/fork work; Stage 4 is research. Throughput figures are engineering targets, not specification guarantees.

scheduler a fixed input. There is no carry queue—a carry would require turning a queue root into committed state—and skipped transactions are recovered by re-inclusion since their nonces are unchanged.

7.7. Fee model

The base fee is burned (accumulated for audit). The priority fee of each transaction is credited to the `evm_coinbase` declared by the `payload` block that included it—because inclusion is the scarce DAG-wide resource the payload miner provides, whereas execution is a mandatory condition of chain candidacy and needs no extra incentive for the accepting producer. The `COINBASE` opcode returns the accepting chain block’s coinbase (one block, one coinbase). Consequently the priority-fee beneficiary can differ per transaction within an EVM block; this is disclosed as an EVM-execution-environment difference, with the per-transaction beneficiary exposed in a `misaka_feeRecipient` receipt field, while `eth_getBlockByNumber.miner` returns the accepting coinbase.

7.8. Staged scalability

The EVM lane has a four-stage roadmap (Figure 5) with an explicit policy that single-number TPS competition belongs only to the final stage.

Stage 1 (active, v0.4 fork). EVM inclusion is open to all DAG blocks; execution is mergeset delayed acceptance; inclusion latency is the DAG block interval (≈ 100 ms at 10 BPS).

Stage 2 (implementation, no consensus change). Optimistic parallel execution (Block-STM-style) with the GHOSTDAG order fixed; the *only* specification requirement is bit-exact serial equivalence—parallel results must equal serial execution in canonical order. EIP-2930 access lists are used only as scheduler hints (no gas discount). Expected speed-up is workload-dependent: 10–20 $\times$ at low contention, none on a single hot pool.

Stage 3 (v0.5 fork-gated). Deferred state-root commitment: execution still happens at chain-context validation, but the Merkleization and commitment are delayed by k blocks ($\text{state_root}(B) = \text{state_root}(\text{ANCESTOR}(B, k))$), moving Merkleization off the critical path. State moves to a flat key-value store with asynchronous Merkleization; node state distinguishes `canonical_tip`, `executed_tip`, and `verified_tip = canonical_tip - k`, with a `misaka_verified` RPC tag and a mismatch rollback/disqualification protocol. The achievable gas-per-second is bounded by $G_{\text{sec}}^{\text{max}} = \text{REPLAY_SPEED}/\text{SAFETY_MARGIN}$ (10–20 $\times$).

Stage 4 (research). An enshrined validity-proof (zkEVM) lane that replaces full re-execution with proof verification, the only path to single-number TPS competition; targets are proof latency ≤ 10 s, 128-bit security, proof size ≤ 300 KiB, with specified prover-liveness, decentralization, data-availability, and forced-inclusion requirements.

Invariant	
I1	Mergeset partition: each EVM user tx executes at most once per selected chain.
I2	EVMRESULT(B) depends only on B 's parents and B 's system ops, not its user payload.
I3	A virtual change is a pointer update only (no re-execution).
I4	Skips (classes 2/3/5) do not affect state, receipts, gas used, or nonces.
I5	Parallel execution equals canonical-order serial execution bit-for-bit.
I6	Priority-fee transfers + burn + balance changes satisfy the supply invariant.
I7	A withdrawal's synthetic UTXO appears only in the accepting block's own UTXO diff.
I8	<code>evm_timestamp_sec</code> is non-decreasing along the selected chain.
I9	Neither the current L1 nor EVM block hash is an EVM input.
I10	The default node build (EVM feature off) links no secp256k1.

Table 5. EVM-lane consensus invariants.

7.9. PQ-only integration and secp isolation

The EVM lane reintroduces secp256k1/ECDSA (`ecrecover` and signature verification) as an *independent* signature domain. Integration with the PQ-only invariant is enforced structurally: the EVM *types* (in consensus-core) are always compiled and secp-free; the EVM *executor* (`revm` and its k256 dependency) sits behind a cargo `evm` feature (default off); the default `kaspad` stays secp-free, enforced by the CI guard across the production binaries; and a non-EVM binary that joins an EVM-active network halts explicitly rather than silently forking. The EVM lane's quantum risk is disclosed as equal to Ethereum's; the post-quantum guarantee covers the UTXO lane only.

7.10. Consensus invariants

The lane is specified by ten invariants (Table 5); they are the contract a conforming implementation must keep and the basis of the conformance tests.

7.11. Performance-claims discipline

MISAKA separates *specification-guaranteed* values (derivable from consensus parameters) from *engineering targets* (benchmark-backed, cited with conditions), and forbids advertising a single hot-object number as general throughput. Latency is always reported in three layers rather than as one “100 ms” headline (Table 6): data-availability inclusion ($\approx$ DAG block interval), the execution receipt ($\approx$ selected-chain interval), and probabilistic finality (blue-work confirmation depth, exposed as `safe/finalized`). The positioning relative to other parallel systems is bounded: the UTXO lane is the owned-object/fastpath analogue and the EVM lane the shared-state/consensus-path analogue, and MISAKA does not claim a single-number crown for a fully-re-executed EVM. As context, transfer throughput at a 1–3 Mgas/s setting is on the order of 60–140 TPS, rising toward 5,000–24,000 TPS with parallel execution and a flat state backend; these are engineering targets in the sense above, validated against a ten-workload benchmark matrix (native transfer, ERC-20 with disjoint balances, hot-token disjoint balances, single and many Uniswap pools, concentrated liquidity, liquidation hotspot, dependent bundles, adversarial skip-flood, and anchor-to-tip replay).

8. The UTXO $\leftrightarrow$ EVM Bridge

The two lanes share one native coin. Value crosses between them through an in-consensus deposit/withdraw mechanism—no external custodian, no wrapped IOU—that conserves a single supply (Figure 6).

Quantity	Class	Value	Condition
DA-inclusion latency	spec-guaranteed	$\approx$ DAG block interval	BPS-dependent
Receipt latency	spec-guaranteed	$\approx$ selected-chain interval	effective-width dependent
Stage 2 throughput	engineering target	20–100 Mgas/s	low–mid contention
Stage 3 throughput	engineering target	100–500 Mgas/s	flat backend, NVMe
Transfer TPS	engineering target	gas/s $\div$ 21,000	many independent senders
Uniswap swap TPS	engineering target	gas/s $\div$ 120–150k	<i>many pools only</i>

Table 6. Performance-claims matrix. Specification-guaranteed values follow from consensus parameters; engineering targets are benchmark-backed and condition-qualified.

8.1. The supply invariant

At all times,

$$\text{UTXO_balances} + \text{EVM_deposit_locks} + \text{EVM_native_balances} + \text{burned} = \text{issued}. \quad (10)$$

Equation (10) is checked in $O(1)$ from the `evm_total_native_balance` accumulator carried in the execution header (Section 7); priority-fee transfers, base-fee burns, and balance changes all preserve it (invariant I6).

8.2. Deposit (two-stage)

1. **Lock.** A user creates an `EVM_DEPOSIT_LOCK` output on the UTXO lane (subnetwork `0x20`, script class `EvmDepositLock`) carrying the destination EVM address, a `timeout_daa_score`, and a `claim_tip_sompi` inclusion incentive.
2. **Claim.** A producer places a `DepositClaim` system operation in its block B ’s payload; the claim is validated against the *selected-parent* UTXO view of B (so a lock created in the same block is not yet claimable). On success it credits `amount_sompi  $\times$  EVM_NATIVE_SCALE` to the destination and consumes the lock in B ’s own per-block UTXO diff.
3. **Refund.** After the timeout, the user reclaims the lock with an ordinary spend.

The exclusive window—a claim is valid iff `accepting_block_daa_score < timeout_daa_score`—means there is no DAA region in which claim and refund are simultaneously valid. Claims are bounded per EVM block (at most 256 claims, 64 KiB of payload, and a system-gas ceiling at 25,000 gas/claim); a violation disqualifies the block.

8.3. Withdraw (the F002 precompile)

An accepted user transaction calls the precompile at `0x...F002`, `withdraw(bytes script_public_key, uint256 amount_wei)`. User-input faults—malformed calldata, a zero amount, an `amount_wei` that is not an exact multiple of `EVM_NATIVE_SCALE`, a script-rule violation, or insufficient balance—revert as a class-4 transaction, leaving the block valid. On success the EVM balance is debited and a `WithdrawOp` is emitted. Although the withdrawing transaction’s payload came from another block, execution happens in the accepting block B , so the synthetic UTXO output is materialized in B ’s *own* per-block UTXO diff (invariant I7); its outpoint is derived deterministically from a frozen domain and is unspendable within the same block. Because it rides the per-block diff, a reorganization rolls it back automatically through the existing reversed-diff path.

9. Native Coin and Emission

The native coin is MSK, denominated in sompi (1 MSK = 10^8 sompi). The supply is capped at 30 billion MSK: a 15 billion premine plus 15 billion of network emission over twenty years.

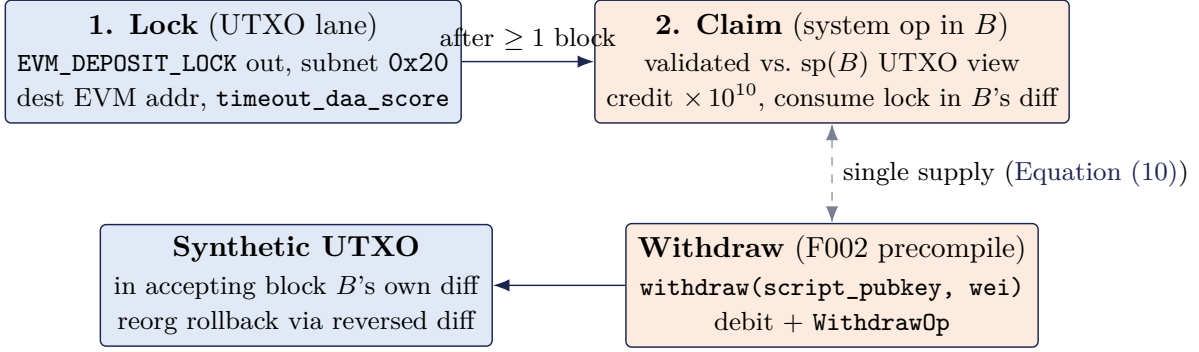


Figure 6. The UTXO ↔ EVM bridge. Deposits are a two-stage lock-then-claim with an exclusive refund window; withdrawals burn EVM balance and materialize a synthetic UTXO in the accepting block’s own diff, so reorganizations roll them back for free.

Year	Per-block subsidy at 10 BPS (MSK)	Annual issuance (MSK)
1	3.70468	$\approx 1.169 \times 10^9$
2	3.51945	$\approx 1.111 \times 10^9$
5	3.01749	$\approx 9.52 \times 10^8$
10	2.33487	$\approx 7.37 \times 10^8$
20	1.39798	$\approx 4.41 \times 10^8$
> 20	0	0

Table 7. Native-coin emission (5%/yr exponential decay, twenty years), derived from Equation (11). Values shown are representative table points; issuance terminates after year 20.

Premine. The premine is a single 15-billion-MSK output to one ML-DSA-87 P2PKH key, committed by the genesis UTXO commitment. On mainnet the premine owner is a placeholder (an all-zero, unspendable payload) until a custody ceremony fixes the real key; that ceremony re-generates the mainnet hash, and a launch-safety flag marks the ceremony as pending. Test networks use a publicly-recoverable premine key so anyone can fund testing.

Emission schedule. Network emission decays 5% per year for twenty years, then stops. With E_y the year- y issuance,

$$E_y = E_1 \cdot 0.95^{y-1}, \quad E_1 = 15 \times 10^9 \cdot \frac{1 - 0.95}{1 - 0.95^{20}} \approx 1.169 \times 10^9 \text{ MSK/yr.} \quad (11)$$

The per-second subsidy is a precomputed 241-entry monthly table; the coinbase manager divides each entry by the actual BPS. At 10 BPS the year-1 rate is ≈ 3.70468 MSK per block (3,704,683,450 sompi/s before the BPS divide), the year-over-year ratio is 0.95 to within 10^{-4} , and issuance is exactly zero from month 240 onward, summing to ≈ 15 billion MSK over the twenty years (Table 7).

Reward routing. Block rewards fan out to attesting validators under the inclusion economics of the finality overlay (a worker-inclusion sub-pool of the subsidy funds attestation inclusion), reusing the coinbase fan-out mechanism while the amount model follows the quality-gated design of Section 6. On the EVM lane, the base fee is burned, which removes issuance from circulation in proportion to lane usage.

10. Fact Settlement Layer (Roadmap)

The Fact Settlement Layer (FSL) is a *proposed* subsystem (design v0.3, economics v0.5); it is not implemented, and its only consensus change is one EVM precompile. We summarize it

because it motivates the 0x...F003 ML-DSA-87 verify precompile reserved in Table 3.

FSL is deliberately *not* a prediction market but an evidence-anchored mechanism for settling machine-checkable facts. Four principles drive it. First, *kill ambiguity at authoring time*: a claim is written in a Predicate DSL whose `predicate_hash` is fixed at creation, with no post-hoc edit path, so the “event time vs. confirmation time” class of disputes is excluded structurally. Second, *mechanize the adjudication hierarchy* as an escalation ladder—evidence > reputation > expertise > stake > vote—in which a vote that cites no evidence is invalid and a permanent void escape hatch exists. The ladder has five rungs: L0 automated source resolution (only when a template permits, with a mandatory challenge window), L1 optimistic resolution (propose with a bond, challenge window), L2 panel verdict, L3 stake-weighted court verdict and appeal (a last resort), and L4 void; the state flow is `Created` → `Proposed` → `Settled`, branching to `Disputed` → `PanelVerdict` → `CourtVerdict` or to `Voided`. Third, *verifiable accountability*: a verdict carries a third-party-recomputable forensic evidence package (with parser id/version and extracted fields, bodies stored in an archival network under a pin obligation plus storage challenge and slashing), participation is *entity-bound* (one entity, one seat), and economic guarantees are published as *decomposed* risk rather than a single guarantee number. Fourth, *earn credibility by record*: ship an Evidence Graph API with no settlement power first, build a methodology-fixed public Shadow Resolution track record, then layer the Settlement Adapter on top.

FSL reuses the DNS finality overlay for its adjudicator panel and its juror-selection beacon (an attester commit-reveal beacon, never `prevrandao`), and uses ML-DSA-87 plus Hash64 as the PQ-safe domain for evidence and verdict attribution. Source reputation scores are informative, not authoritative (illustrative initial values: official 980, exchange 950, news agency 900, registered 400, anonymous 100): they gate L0 admissibility, bond discounts, and display only, while a court judges evidence *de novo*. The economic design adds no new token. The honest limit, stated plainly in the design, is that the oracle problem cannot be solved, only structured: FSL settles “what the protocol adjudicated, with an evidence trail,” not metaphysical truth.

11. Implementation and Network Status

MISAKA is a fork of *rusty-kaspa*, the Rust Kaspa full node. The node binary keeps the name `kaspad` and the crates keep their upstream `kaspa-*` names—this is a fork, not a rename—while the network, the addresses (`misaka/misakatest/misakadev`), and the branding are MISAKA’s. It is distributed under the same ISC license as upstream, and all upstream credit is due to the Kaspa project.

The live network is an experimental `testnet-10`, with a block explorer at `misakascan.com` and DNS seeders `seeder1.misakascan.com` / `seeder2.misakascan.com`; its P2P port is 26211 (mainnet 26111, devnet 26611). PQ-only consensus and the DNS-finality reward overlay are active from genesis on every defined network (`pq_activation_daa_score = 0`, `dns_activation_daa_score = 0`), and the testnet/mainnet parameter sets additionally enforce the production finality policy (two-dimensional confirmation, 20M-MSK minimum bond). The EVM lane was activated on testnet on 2026-06-11 via a three-host re-genesis cutover that left the testnet genesis hash unchanged (`cf4c48fe...`); post-activation headers carry `EVM_HEADER_VERSION 2`, so pre-EVM nodes and the old chain are version-isolated in both directions, and EVM-transaction relay is back-compatible across the protocol version bump. The mainnet parameter set is **defined but not launched or endorsed for production**.

The tree ships the node, a miner, the validator sidecar and its remote signer, a DNS seeder, the bridge, a PQ command-line wallet, a WASM SDK, and a browser-extension wallet. The continuous-integration guard hard-fails if the consensus crate or the node links `secp256k1`, and the EVM executor is reachable only via the `evm` build feature.

12. Security Considerations

Scope of the post-quantum claim. The PQ guarantee covers transaction authorization (ML-DSA-87), the consensus identity (Hash64), and the UTXO accumulator (lattice LtHash32). It does *not* cover transport confidentiality, which is post-quantum only if an ML-KEM hybrid key exchange is enabled—out of scope at launch and disclosed as such. Legacy Kasper addresses are not quantum-resistant and are unrepresentable here by construction.

The EVM lane is not post-quantum. The EVM signature domain is secp256k1, with the same quantum exposure as Ethereum; this is disclosed, and the post-quantum guarantee is explicitly the UTXO lane's. The lane is isolated behind a build feature so the default node never links secp256k1, and a non-EVM binary halts rather than silently forks on an EVM-active network.

Reorg resistance under a compute-only PoW. Because Phase-3 PoW is not ASIC-egalitarian, single-resource (hashing-power) dominance is more plausible than under a memory-hard PoW. The DNS overlay is the mitigation: a deep reorganization that exits a stake-confirmed anchor must clear the two-dimensional dominance gate of Equation (6), which a pure-PoW majority cannot do without also dominating stake. The quality floor φ_S denies a minority stake the ability to accumulate score over a long window.

Long-range and equivocation. The unbonding bound $U \geq R + E$ keeps a bond slashable for as long as any equivocation it signed can be submitted; the slashing-evidence object burns a double-signing bond within the evidence window; and the remote signer's strict policy adds a crash-consistent anti-equivocation guard outside the validator process.

Denial of service and propagation. ML-DSA objects are large, so mass is recalibrated to price signature verification and cap per-block compute, and the EVM lane's byte and gas budgets are separate from the UTXO mass budget so a full EVM payload cannot degrade UTXO throughput. Because EVM payloads enlarge blocks, EVM activation is gated on re-validating the propagation-safety inequality Equation (5) with payload-inclusive measured delay.

Dependency posture. The post-quantum safety of the signature dependency is not static; the build pins `libcrux-ml-dsa` and runs advisory and audit checks in continuous integration, treating the lattice library as a monitored surface rather than a fixed assumption.

13. Related Work

MISAKA's DAG engine is Kasper's GHOSTDAG [1], from which it inherits block production, blue-work tip selection, and the difficulty-adjustment algorithm; the contribution here is the post-quantum migration, the finality overlay, and the EVM lane, not the DAG protocol. The signature, hash, and KEM standards are the NIST post-quantum suite: ML-DSA (FIPS 204) for signatures [2], with ML-KEM (FIPS 203) [3] the hybrid-transport option and SLH-DSA (FIPS 205) [4] a stateless-hash alternative; the symmetric primitives are BLAKE2 [6] and SHA-3 [5]. The lattice UTXO accumulator follows the additive-homomorphic LtHash line [7].

The EVM lane's execution model draws on the parallel-execution and deferred-state literature: Block-STM optimistic concurrency with a fixed preset order [8], and Monad-style asynchronous execution with delayed Merkle roots [9], adapted so that execution still occurs at chain-context validation and only commitment is deferred. The performance positioning is stated against Sui's owned-versus-shared object execution paths [10], deliberately avoiding a single-number comparison for re-executed EVM and reserving that competition for a future validity-proof lane

in the manner of zk-rollups [11] and real-time L1 zkEVM proving targets. The EVM-environment conventions follow the Ethereum standards for chain-id replay protection, optional access lists, and the fee market [12, 13, 14]. The Fact Settlement Layer’s framing as evidence-anchored adjudication is a response to the documented failure modes of optimistic token-vote oracles.

14. Conclusion and Future Work

MISAKA shows that a single base layer can be post-quantum on its consensus surface, retain a high-throughput BlockDAG, and host a consensus-native EVM, provided the pieces are chosen to reinforce one another rather than merely coexist. Making transaction authorization ML-DSA-87, the consensus identity a 64-byte keyed BLAKE2b-512, and the UTXO accumulator a lattice LtHash closes the quantum-margin asymmetry on the UTXO lane. Adding a two-resource DNS finality overlay is what makes a fast, compute-only proof-of-work acceptable, because deep history rewriting then requires dominating stake as well as work. And binding the EVM parent to the GHOSTDAG selected parent, with Kaspas-congruent mergeset delayed acceptance, lets an inherently-sequential EVM live on a reorganizing DAG while keeping “immutable per block hash, pointer-switch on reorg” true—no external bridge, no sequencer.

Future work follows the staged roadmap: serial-equivalent parallel execution (Stage 2), deferred state-root commitment with a flat backend (Stage 3), and a research-stage enshrined validity-proof lane (Stage 4) as the only honest path to single-number throughput; the Fact Settlement Layer and its ML-DSA-87 verify precompile; and, on the cryptographic side, an optional ML-KEM hybrid transport to extend the post-quantum claim to confidentiality. Throughout, the project holds to an explicit discipline separating what the consensus parameters guarantee from what benchmarks target. MISAKA builds on the Kaspas project’s BlockDAG, to whom all upstream credit is due.

References

- [1] Y. Sompolinsky, S. Wyborski, and A. Zohar. *PHANTOM GHOSTDAG: A Scalable Generalization of Nakamoto Consensus*. Cryptology ePrint Archive, Report 2018/104; and the Kaspas project documentation, wiki.kaspas.org.
- [2] National Institute of Standards and Technology. *FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)*. 2024.
- [3] National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*. 2024.
- [4] National Institute of Standards and Technology. *FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA)*. 2024.
- [5] National Institute of Standards and Technology. *FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. 2015.
- [6] M.-J. Saarinen and J.-P. Aumasson. *The BLAKE2 Cryptographic Hash and Message Authentication Code (MAC)*. RFC 7693, 2015.
- [7] Meta / R. Lewi et al. *Securing Update Propagation with Homomorphic Hashing* (LtHash, additive lattice accumulator). Cryptology ePrint Archive, Report 2019/227.
- [8] R. Gelashvili et al. *Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing*. arXiv:2203.06871, 2022.
- [9] Monad Labs. *Asynchronous Execution and Deferred Merkle-Root Commitment*. Monad documentation, docs.monad.xyz.
- [10] Mysten Labs. *Sui: Owned- and Shared-Object Execution Paths*. Sui documentation and performance reports, docs.sui.io.

- [11] Ethereum Foundation. *Zero-Knowledge Rollups and Shipping an L1 zkEVM: Realtime Proving*. ethereum.org, 2025.
- [12] V. Buterin. *EIP-155: Simple Replay Attack Protection*. Ethereum Improvement Proposals, 2016.
- [13] V. Buterin and M. Swende. *EIP-2930: Optional Access Lists*. Ethereum Improvement Proposals, 2020.
- [14] V. Buterin et al. *EIP-1559: Fee Market Change for ETH 1.0*. Ethereum Improvement Proposals, 2019.